

Specification of Source §3 Type Inference—2021 edition

Martin Henz, K Muruges, Raynold Ng, Daryl Tan, Tse Hiu Fung

National University of Singapore
School of Computing

July 1, 2026

1 Notation

1.1 The language Source §3

The set of expressions E is the least set that satisfies the following rules, where x ranges over a set of names V , n ranges over the positive integers, p_1 ranges over the set of unary primitive operations $P_1 = \{!\}$, and p_2 ranges over the set of binary primitive operations $P_2 = \{||, \&\&, +, -, *, /, \%, ==, !=, >, <, <=, >=\}$.

x	i	s	true	false	undefined
E	$E_1 \ E_2$	$E \ E_1 \ E_2$	$E \ E_1 \ \dots \ E_n$		
$p_1[E]$	$p_2[E_1, E_2]$	$E \ ? \ E_1 : E_2$	$E \ (\ E_1, \dots, E_n)$		
$E_1 \ E_2 \ \dots \ E_n$				$E \ E_1$	
$[E_1, E_2, \dots, E_n]$				$E[E_1]$	

The letters x , i and s stand for names, numbers, strings, respectively. The set of possible assignments $assign$ is the least set that satisfies the two following rules, representing variable and array assignment.

E	$E \ E_1 \ E_2$
$x = E$	$E[E_1] = E_2$

Let the set of all expressions (the union of R and E) be denoted by U . Also let the term **decl** denote the two keywords for variable declaration: **let** and **const**. The set of all statements S is then the least set that satisfies the following rules.

E	E	E	$assign$	S
decl $x = E ;$	return $E ;$	$E ;$	$assign ;$	function $f(x_1, \dots, x_n) \{ S \}$
$S_1 \ S_2$	$\{ S \}$	if $(E) \{ S_1 \}$ else $\{ S_2 \}$	while $(E) \{ S \}$	
$assign_1 \ assign_2 \ E \ S$	$E_1 \ assign \ E_2 \ S$			
for $(assign_1 ; E ; assign_2) \{ S \}$	for $(\text{let } x = E_1 ; E_2 ; assign) \{ S \}$	break ;		

continue ;

We introduce the following additional rule for expressions, in order to define functions.

$$\frac{S}{(x_1, \dots, x_n) \Rightarrow \{ S \}}$$

We treat function declaration statements of the form

function $f(x_1, \dots, x_n) \{ S \}$

as abbreviations for constant declaration statements as follows

const $f = (x_1, \dots, x_n) \Rightarrow \{ S \};$

function definitions of the form

$(x_1, \dots, x_n) \Rightarrow E$

as abbreviations for the following

$(x_1, \dots, x_n) \Rightarrow \{ \textbf{return } E ; \}$

Conditional statements of the form

```
if (x1) {
  const x = 1;
} else if (x2) {
  const y = 3;
} else if (x3) {
  const a = 3;
} else {
  const b = 3;
}
```

are treated as abbreviations for the following

```
if (x1) {
  const x = 1;
} else {
  if (x2) {
    const y = 3;
  } else {
    if (x3) {
      const a = 3;
    } else {
      const b = 3;
    }
  }
}
```

For statements in the following form (having a declaration as the initialization)

```
for (let x = 4; x < 20; x = x + 1) {
  ...
}
```

are treated as abbreviations for the following

```
{
  let x = 4;
  for (x = x; x < 20; x = x + 1) {
    ...
  }
}
```

1.2 A Language of Types

We introduce the following language of types for type inference:

T_i type		A_i type	
number type		bool type	string type
undefined type			
t_1 type	$\dots$	t_n type	t type
$(t_1, \dots, t_n) \rightarrow t$ type		t_{head} type	t_{tail} type
$\text{Pair}(t_{\text{head}}, t_{\text{tail}})$ type		t	t
t type		t polytype	t type
$\forall(t)$ polytype		$\text{Pred}(t)$ predtype	$\text{Pred}(t)$ predtype

where $n \geq 1$, and T_i and A_i represent type variables. We will capitalize type variables, as in T_1, A_2 . We will also refer to the types in the second row (i.e. `bool`, `undefined`, `number`, `string`) as *base types*. The symbols t_i in the rules above are meta-variables that stand for types and must not be confused with type variables that *are* types. As usual, parentheses can be used in practice for grouping. Examples of valid types are `number` and `(number, ()  $\rightarrow$  bool, undefined,  $T_1$ )  $\rightarrow$  (bool  $\rightarrow$   $A_2$ )`.

Types of the form $\text{Pred}(t)$ are called *predicate types*, types of the form $\forall(t)$ are called *polymorphic types*, and all other are called *monomorphic types*.

We distinguish two kinds of type variables, T_i and A_i , to be able to handle the overloading of operators such as `+` (for numbers and strings). A type variable A_i can only represent “addable” types, i.e. `number` or `string`, and a type variable T_i can represent any type.

1.3 Type Environments

For Source, well-typedness of an statement depends on the context in which the statement appears. The expression `x + 3` within a statement may or may not be well-typed, depending on the type of `x`. Thus in order to formalize the notion of a context, we define a *type environment*, denoted by Γ , that keeps track of the type of names appearing in the statement. More formally, the partial function Γ from names to types expresses a context, in which a name x is associated with type $\Gamma(x)$.

We define a relation $\Gamma[x \leftarrow t]\Gamma'$ on type environments Γ , names x , types t , and type environments Γ' , which constructs a type environment that behaves like the given one, except that the type of x is t . More formally, if $\Gamma[x \leftarrow t]\Gamma'$, then $\Gamma'(y)$ is t , if $y = x$ and $\Gamma(y)$ otherwise. Obviously, this uniquely identifies Γ' for a given Γ , x , and t , and thus the type environment extension relation is functional in its first three arguments.

The set of names, on which a type environment Γ is defined, is called the domain of Γ , denoted by $\text{dom}(\Gamma)$.

For each nonoverloaded primitive operator, we add a binding to our initial type environment Γ_0 as follows:

$$\begin{aligned}
&\emptyset[-_2 \leftarrow (\text{number}, \text{number}) \rightarrow \text{number}] \\
&[* \leftarrow (\text{number}, \text{number}) \rightarrow \text{number}] \\
&[/ \leftarrow (\text{number}, \text{number}) \rightarrow \text{number}] \\
&[\% \leftarrow (\text{number}, \text{number}) \rightarrow \text{number}] \\
&[\&\& \leftarrow \forall((\text{bool}, \text{bool}) \rightarrow \text{bool})] \\
&[|| \leftarrow \forall((\text{bool}, \text{bool}) \rightarrow \text{bool})] \\
&[! \leftarrow \text{bool} \rightarrow \text{bool}] \\
&[-_1 \leftarrow \text{number} \rightarrow \text{number}]\Gamma_{-2}
\end{aligned}$$

The overloaded binary primitive are handled as follows:

$$\begin{aligned}
&\Gamma_{-2}[+ \leftarrow \forall((A, A) \rightarrow A)] \\
&[=== \leftarrow \forall((T_1, T_2) \rightarrow \text{bool})] \\
&[! == \leftarrow \forall((T_1, T_2) \rightarrow \text{bool})] \\
&[> \leftarrow \forall((A, A) \rightarrow \text{bool})] \\
&[> = \leftarrow \forall((A, A) \rightarrow \text{bool})] \\
&[< \leftarrow \forall((A, A) \rightarrow \text{bool})] \\
&[< = \leftarrow \forall((A, A) \rightarrow \text{bool})]\Gamma_{-1}
\end{aligned}$$

Γ_{-1}	[	display	$\leftarrow$	$\forall(T)$	]
	[	error	$\leftarrow$	$\forall(T)$	]
	[	Infinity	$\leftarrow$	number	]
	[	is_boolean	$\leftarrow$	$Pred(\text{bool})$	]
	[	is_function	$\leftarrow$	$Pred(\forall(T_1 \rightarrow T_2))$	]
	[	is_number	$\leftarrow$	$Pred(\text{number})$	]
	[	is_string	$\leftarrow$	$Pred(\text{string})$	]
	[	is_undefined	$\leftarrow$	$Pred(\text{undefined})$	]
	[	math_abs	$\leftarrow$	number $\rightarrow$ number	]
	[	math_acos	$\leftarrow$	number $\rightarrow$ number	]
	[	math_acosh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_asin	$\leftarrow$	number $\rightarrow$ number	]
	[	math_asinh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_atan	$\leftarrow$	number $\rightarrow$ number	]
	[	math_atan2	$\leftarrow$	(number, number) $\rightarrow$ number	]
	[	math_atanh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_cbrt	$\leftarrow$	number $\rightarrow$ number	]
	[	math_ceil	$\leftarrow$	number $\rightarrow$ number	]
	[	math_clz32	$\leftarrow$	number $\rightarrow$ number	]
	[	math_cos	$\leftarrow$	number $\rightarrow$ number	]
	[	math_cosh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_exp	$\leftarrow$	number $\rightarrow$ number	]
	[	math_expm1	$\leftarrow$	number $\rightarrow$ number	]
	[	math_floor	$\leftarrow$	number $\rightarrow$ number	]
	[	math_fround	$\leftarrow$	number $\rightarrow$ number	]
	[	math_hypot	$\leftarrow$	$\forall(T)$	]
	[	math_imul	$\leftarrow$	(number, number) $\rightarrow$ number	]
	[	math_LN2	$\leftarrow$	number	]
	[	math_LN10	$\leftarrow$	number	]
	[	math_log	$\leftarrow$	number $\rightarrow$ number	]
	[	math_log1p	$\leftarrow$	number $\rightarrow$ number	]
	[	math_log2	$\leftarrow$	number $\rightarrow$ number	]
	[	math_LOG2E	$\leftarrow$	number	]
	[	math_log10	$\leftarrow$	number $\rightarrow$ number	]
	[	math_LOG10E	$\leftarrow$	number	]
	[	math_max	$\leftarrow$	$\forall(T)$	]
	[	math_min	$\leftarrow$	$\forall(T)$	]
	[	math_PI	$\leftarrow$	number	]
	[	math_pow	$\leftarrow$	(number, number) $\rightarrow$ number	]
	[	math_random	$\leftarrow$	() $\rightarrow$ number	]
	[	math_round	$\leftarrow$	number $\rightarrow$ number	]
	[	math_sign	$\leftarrow$	number $\rightarrow$ number	]
	[	math_sin	$\leftarrow$	number $\rightarrow$ number	]
	[	math_sinh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_sqrt	$\leftarrow$	number $\rightarrow$ number	]
	[	math_SQRT1_2	$\leftarrow$	number	]
	[	math_SQRT2	$\leftarrow$	number	]
	[	math_tan	$\leftarrow$	number $\rightarrow$ number	]
	[	math_tanh	$\leftarrow$	number $\rightarrow$ number	]
	[	math_trunc	$\leftarrow$	number $\rightarrow$ number	]
	[	NaN	$\leftarrow$	number	]
	[	parse_int	$\leftarrow$	(string, number) $\rightarrow$ number	]
	[	prompt	$\leftarrow$	string $\rightarrow$ string	]
	[	runtime	$\leftarrow$	() $\rightarrow$ number	]
	[	stringify	$\leftarrow$	$\forall(T) \rightarrow \text{string}$	]
	[	undefined	$\leftarrow$	undefined	]

[null	←	$\forall(List(T))$			]
[pair	←	$\forall((T_1, T_2)$	→	$Pair(T_1, T_2))$	]
[head	←	$\forall(Pair(T_1, T_2)$	→	T_1	]
[tail	←	$\forall(Pair(T_1, T_2)$	→	T_2	]
[list	←	$\forall(T_1, T_2, \dots, T_n$	→	$List(T_{n+1}))$	]
[is_pair	←	$Pred(\forall(Pair(T_1, T_2)))$			]
[is_null	←	$Pred(\forall(List(T)))$			]
[is_list	←	$Pred(\forall(List(T)))$			]
[set_head	←	$\forall(Pair(T_1, T_2)$	→	undefined)	]
[set_tail	←	$\forall(Pair(T_1, T_2)$	→	undefined)	]
[is_array	←	$Pred(\forall(Array(T)))$			]
[array_length	←	$\forall(Array(T) \rightarrow \text{number})$			]

] Γ_0

1.4 Preparing Programs for Type Inference

To facilitate the process of type inference, we annotate each component of the given program with unique type variables and introduce a simple transformation at the toplevel.

A *toplevel transformation* clarifies the nature of the names declared outside of function definitions, and the type of the overall statement. The toplevel transformation wraps the given program into a block, and introduces **return** keywords in front of expression statements, when these are the last statements in a sequence to be evaluated, even when they occur within conditional statements.

Examples:

```
const x = 1;
x + 2;
```

becomes

```
{
  const x = 1;
  return x + 2;
}
```

and

```
if (true) {
  const x = 1;
  x + 2;
} else {
  const y = 3;
  y + 4;
}
```

becomes

```
{
  if (true) {
    const x = 1;
    return x + 2;
  } else {
    const y = 3;
    return y + 4;
  }
}
```

To facilitate the process of type inference, we annotate each component of the given program with unique type variables. We write the type variable as a superscript after the component, and use parentheses for clarification. For example, the Source §1 program

```
{ const x = 1; return x + 2; }
```

is represented by the annotated program

$$\text{const } x^{T_1} = 1^{T_2}; \text{ return } (x^{T_3} + 2^{T_4})^{T_5};$$

1.5 Type Constraints

We introduce type constraints Σ as conjunctions of type equations:

$$\frac{}{\top} \qquad \frac{}{t_1 = t_2} \qquad \frac{\Sigma_1 \quad \Sigma_2}{\Sigma_1 \wedge \Sigma_2}$$

We require that constraints are kept in *solved form*:

$$t_1 = t'_1 \wedge \dots \wedge t_i = t'_i \wedge \dots \wedge t_n = t'_n$$

where:

- all t_i are type variables,
- for any type variable T_i , there is at most one equation $T_i = \dots$,
- no variable t_i occurs in any equation $t_j = t'_j$ if $j > i$.

A constraint in solved form does not have any cycles $t^{(0)} = t^{(1)}, t^{(1)} = t^{(2)}, \dots, t^{(k)} = t^{(0)}$. We *apply* a type constraint Σ in solved form to a type t as follows:

$$\frac{\text{if } t_i \text{ is a base type or } t_i = t'_i \text{ does not occur in } \Sigma}{\Sigma(t_i) = t_i} \qquad \frac{\text{if } t_i = t'_i \text{ occurs in } \Sigma}{\Sigma(t_i) = \Sigma(t'_i)}$$

$$\frac{t' = \Sigma(t) \quad t'_1 = \Sigma(t_1) \quad \dots \quad t'_n = \Sigma(t_n)}{\Sigma((t_1, \dots, t_n) \rightarrow t) = (t'_1, \dots, t'_n) \rightarrow t'} \qquad \frac{t' = \Sigma(t)}{\Sigma(\text{List}(t)) = t'}$$

$$\frac{t'_{\text{head}} = \Sigma(t_{\text{head}}) \quad t'_{\text{tail}} = \Sigma(t_{\text{tail}}) \quad t'_{\text{tail}} = \text{List}(t'_{\text{head}})}{\Sigma(\text{Pair}(t_{\text{head}}, t_{\text{tail}})) = t'_{\text{tail}}}$$

$$\frac{t'_{\text{head}} = \Sigma(t_{\text{head}}) \quad t'_{\text{tail}} = \Sigma(t_{\text{tail}}) \quad t'_{\text{tail}} \neq \text{List}(t'_{\text{head}})}{\Sigma(\text{Pair}(t_{\text{head}}, t_{\text{tail}})) = \text{Pair}(t'_{\text{head}}, t'_{\text{tail}})}$$

Example: If $\Sigma = (T_1 = \text{number} \wedge T_2 = T_3 \rightarrow \text{bool} \wedge T_3 = \text{number} \rightarrow \text{bool})$, we have $\Sigma(\text{number} \rightarrow T_2) = \text{number} \rightarrow ((\text{number} \rightarrow \text{bool}) \rightarrow \text{bool})$.

Note that in our framework, type constraints never contain any polymorphic types. Thus you will never see “ $\forall$ ” in a type constraint.

We add a constraint $t = t'$ to a solved form Σ by applying the following rules in the given order:

- If t is a *base type* and t' is also a *base type* of the same kind, do nothing.
- If t is not a type variable and t' is a type variable, then we now try to add $t' = t$ to Σ , following the same rules.
- If t is a type variable and $\Sigma(t')$ is a type variable with the same name as t , do nothing.
- If t is a type variable and $\Sigma(t') = \text{Pair}(t'', t)$, we now try to add the equation $t = \text{List}(t'')$ to Σ , following the same rules.
- If t is a type variable and $\Sigma(t') = \text{Pair}(t'', \text{List}(t'''))$, we now try to add the equations $t'' = t'''$ and $t = \text{List}(t'')$ to Σ , following the same rules.

- If t is a type variable, $\Sigma(t')$ is a function type, list type, or pair type, and t is contained in $\Sigma(t')$, then stop with a type error as we will have an infinite type. (e.g. $A = B \rightarrow A$)
- If t is A_i and $\Sigma(t')$ is not a type variable and not `number` or `string`, then stop with a type error.
- If t is a type variable and there is an equation $t = t''$ in Σ , then we now try to add the equation $t' = t''$ to Σ , following the same rules.
- If t is a type variable that does not occur on the left in any equation in Σ , then add $t = \Sigma(t')$ in the front of Σ . In addition, if $\Sigma(t)$ is an "addable" type variable A_i and $\Sigma(t')$ is a regular type variable T_j , we must convert $\Sigma(t')$ into an "addable" type A_j .
- If t is $(t_1, \dots, t_n) \rightarrow t''$ and t' is $(t'_1, \dots, t'_n) \rightarrow t'''$, then add n constraints $t_1 = t'_1, \dots, t_n = t'_n, t'' = t'''$ to Σ , each time going through the above set of rules.
- If t is $\text{Pair}(t_{\text{head}}, t_{\text{tail}})$ and t' is $\text{Pair}(t'_{\text{head}}, t'_{\text{tail}})$, then add 2 constraints $t_{\text{head}} = t'_{\text{head}}, t_{\text{tail}} = t'_{\text{tail}}$ to Σ , both times going through the above set of rules.
- If t is $\text{List}(t_{\text{el}})$ and t' is $\text{List}(t'_{\text{el}})$, then add the constraint $t_{\text{el}} = t'_{\text{el}}$ to Σ going through the above set of rules.
- If t is $\text{List}(t_{\text{el}})$ and t' is a pair type, then try to add the constraint $t' = \text{Pair}(t_{\text{el}}, t)$ to Σ going through the above set of rules.
- If t' is $\text{List}(t_{\text{el}})$ and t is a pair type, then try to add the constraint $t = \text{Pair}(t_{\text{el}}, t')$ to Σ going through the above set of rules.
- If t is $\text{Array}(t_1)$ and $\Sigma(t') = \text{Array}(t_2)$, add the constraint $t_1 = t_2$ to Σ going through the above set of rules.
- Any other case (e.g. `bool = string`) stops with a type error.

This process is guaranteed to terminate either with a type error or with a new solved form.

2 Typing Relation

The set of well-typed programs is defined by the binary typing relation written $S : \Sigma$, where S is a toplevel-transformed, type-annotated program. The relation is defined using the quaternary typing relation $\Sigma, \Gamma \vdash S : \Sigma'$, as follows: $S : \Sigma$ holds if and only if $\top, \Gamma_0 \vdash S : \Sigma$ where Γ_0 is the initial type environment described above and $\top$ is the empty type constraint. The constraint Σ can be called the constraint *inferred from* S .

We define the typing relation for expressions and statements inductively with the following rules.

2.1 Typing Relation on Expressions

The type of a name needs to be provided by the type environment. The first rule applies when $\Gamma(x)$ is monomorphic, i.e. $\Gamma(x) \neq \forall t'$.

$$\Gamma(x) \text{ type} \quad (\Sigma \wedge t = \Gamma(x)) = \Sigma'$$

$$\Sigma, \Gamma \vdash x^t : \Sigma'$$

If $\Gamma(x)$ is polymorphic, i.e. $\Gamma(x) = \forall t'$, we replace all type variables in t' with fresh type variables:

$$\Gamma(x) \text{ polytype} \quad (\Sigma \wedge t = \text{fresh}(t')) = \Sigma'$$

$$\Sigma, \Gamma \vdash x^t : \Sigma'$$

where $\text{fresh}(t')$ results from t' by replacing all type variables consistently with fresh type variables. Example: $\text{fresh}(\text{bool} \rightarrow (T_1 \rightarrow (T_2 \rightarrow T_2)))$ might return $\text{bool} \rightarrow (T_{77} \rightarrow (T_{88} \rightarrow T_{88}))$.

If $\Gamma(x)$ is a predicate type, i.e. $\Gamma(x) = \text{Pred}(t')$, we treat x as if it had the type $\forall(t_0 \rightarrow \text{bool})$ instead:

$$\frac{\Gamma(x) \text{ predtype} \quad (\Sigma \wedge t = \text{fresh}(t_0) \rightarrow \text{Bool}) = \Sigma'}{\Sigma, \Gamma \vdash x^t : \Sigma'}$$

where $\text{fresh}(t_0)$ is some fresh type variable.

If $\Gamma(x)$ is not defined, then none of the rules are applicable. In this case, we say that there is no type for x derivable from the type environment Γ .

Constants get the following types.

$$\frac{(\Sigma \wedge t = \text{number}) = \Sigma'}{\Sigma, \Gamma \vdash n^t : \Sigma'} \quad \frac{(\Sigma \wedge t = \text{string}) = \Sigma'}{\Sigma, \Gamma \vdash s^t : \Sigma'}$$

where n denotes any literal number s denotes any literal string.

$$\frac{(\Sigma \wedge t = \text{bool}) = \Sigma'}{\Sigma, \Gamma \vdash \text{true}^t : \Sigma'} \quad \frac{(\Sigma \wedge t = \text{bool}) = \Sigma'}{\Sigma, \Gamma \vdash \text{false}^t : \Sigma'}$$

We have the following rule for function application.

$$\frac{\Sigma_0, \Gamma \vdash E_0^{t_0} : \Sigma_1 \quad \dots \quad \Sigma_n, \Gamma \vdash E_n^{t_n} : \Sigma_{n+1} \quad (\Sigma_{n+1} \wedge t_0 = (t_1, \dots, t_n) \rightarrow t) = \Sigma_{n+2}}{\Sigma_0, \Gamma \vdash (E_0^{t_0} (E_1^{t_1}, \dots, E_n^{t_n}))^t : \Sigma_{n+2}}$$

Application of the binary and unary operators are treated in the same way, with the operator being treated as an identifier expression. The type of the operator needs to be a function type with the right number of parameters, and the type of every argument needs to coincide with the corresponding parameter type of the function type. If all these conditions are met, the type of the function application is the same as the return type of the function type that is the type of the operator.

The typing of function definition statements is defined as follows.

$$\frac{\Sigma \wedge (t' = (t_1, \dots, t_n) \rightarrow t), \Gamma[x_1 \leftarrow t_1] \dots [x_n \leftarrow t_n] \vdash S^t : \Sigma'}{\Sigma, \Gamma \vdash ((x_1^{t_1}, \dots, x_n^{t_n}) \Rightarrow \{ S^t \})^{t'} : \Sigma'}$$

For literal array definitions, we require all members of the array to be of the same type.

$$\frac{\Sigma_1, \Gamma \vdash E_1^{t_1} : \Sigma_2 \dots \Sigma_n, \Gamma \vdash E_n^{t_n} : \Sigma_{n+1} \quad (\Sigma_{n+1} \wedge t = \text{Array}(t') \wedge t' = t_1 \wedge t' = t_2 \wedge \dots \wedge t' = t_n) = \Sigma_{n+2}}{\Sigma_1, \Gamma \vdash ([E_1^{t_1}, \dots, E_n^{t_n}])^t : \Sigma_{n+2}}$$

where t' is a fresh type variable. If the array is empty, we simply get:

$$\frac{\Sigma_2 = (\Sigma_1 \wedge t = \text{Array}(t'))}{\Sigma_1, \Gamma \vdash ([])^t : \Sigma_2}$$

Finally, for array member access, we need to infer the property and the object first. We expect the object to be an *Array* and the property to a *number*. One possible type error could occur here at runtime where we try to access an array with a non-integer index. This error cannot be caught by the type inferencer as we do not infer integer types. If we successfully add the two

constraints, we then add a constraint that the type of the entire expression is the element type of the array.

$$\frac{\Sigma_1, \Gamma \vdash E_1^{t_1} : \Sigma_2 \quad \Sigma_2, \Gamma \vdash E^{t_2} : \Sigma_3 \quad (\Sigma_3 \wedge t_1 = \text{number} \wedge t_2 = \text{Array} \wedge t = t_2.\text{elementType}) = \Sigma_4}{\Sigma_0, \Gamma \vdash (E^{t_2}[E_1^{t_1}])^t : \Sigma_3}$$

2.2 Typing Relation on Assignments

For the typing of assignments, the key idea is to infer the types of the LHS and RHS and simply add a constraint that the two are equal. The same rule can be applied for assignment to an identifier and assignment to an array member expression. Collectively, we will just denote the LHS as E_1 to represent both scenarios.

$$\frac{(\Sigma_1 \wedge t = t_0), \Gamma \vdash E^{t_0} : \Sigma_2 \quad \Sigma_2, \Gamma \vdash E_1^{t_1} : \Sigma_3 \quad (\Sigma_3 \wedge t_0 = t_1) = \Sigma_4}{\Sigma_1, \Gamma \vdash (E_1^{t_1} = E^{t_0};)^t : \Sigma_4}$$

2.3 Typing Relation on Statements

The following rule deals with the typing of sequences. We assume that whenever there is a return statement or a conditional statement with a return statement within a sequence, it is the last statement in the sequence. (One could consider a “dead code” error otherwise.)

$$\frac{(\Sigma_1 \wedge t_3 = t_2), \Gamma \vdash S_1^{t_1} : \Sigma_2 \quad \Sigma_2, \Gamma \vdash S_2^{t_2} : \Sigma_3}{\Sigma_1, \Gamma \vdash (S_1^{t_1} S_2^{t_2})^{t_3} : \Sigma_3}$$

Return statements are typed as follows.

$$\frac{(\Sigma \wedge t' = t), \Gamma \vdash E^t : \Sigma'}{\Sigma, \Gamma \vdash (\text{return } E^t;)^{t'} : \Sigma'}$$

The type of expression statements is `undefined`. Note that expression statements at toplevel get a return placed in front of them by the toplevel-transformation described above.

$$\frac{(\Sigma \wedge t' = \text{undefined}), \Gamma \vdash E^t : \Sigma'}{\Sigma, \Gamma \vdash (E^t;)^{t'} : \Sigma'}$$

Similarly, the type of break and continue statements is `undefined`.

$$\frac{(\Sigma \wedge t = \text{undefined}) = \Sigma' \quad (\Sigma \wedge t = \text{undefined}) = \Sigma'}{\Sigma, \Gamma \vdash (\text{break};)^t : \Sigma' \quad \Sigma, \Gamma \vdash (\text{continue};)^t : \Sigma'}$$

For blocks (including the bodies of function definitions), we discern whether the block contains `const` or `let` declarations. If it does not contain any, the typing is easy:

$$S \text{ does not contain } \mathbf{const} \text{ or } \mathbf{let} \quad (\Sigma \wedge t' = t), \Gamma \vdash S^t : \Sigma'$$

$$\Sigma, \Gamma \vdash \{ S^t \}^{t'} : \Sigma_3$$

Blocks (including the bodies of function definitions) that contain declarations introduce polymorphism. In the following rule we assume that S does not have any further declarations. The rule is a simplification of the general case, because statements other than declarations can appear before and between the declarations. The rule applies analogously in this case, without re-arranging the statements. This means that the body of a block has two parts:

- the part up to and including the last declaration, where all declared names are monomorphically typed, and
- the part after the last declaration, where all declared names are polymorphically typed.

$$\begin{array}{l} \Gamma[x_1 \leftarrow t_1] \cdots [x_n \leftarrow t_n] \Gamma' \\ (\Sigma_0 \wedge t = t' \wedge t'_1 = \text{undefined} \wedge \cdots \wedge t'_n = \text{undefined}) = \Sigma_1 \\ \Sigma_1, \Gamma' \vdash E_1^{t_1} : \Sigma_2 \cdots \Sigma_n, \Gamma' \vdash E_n^{t_n} : \Sigma_{n+1} \\ \Gamma'[x_1 \leftarrow \forall \Sigma_{n+1}(t_1)] \cdots [x_n \leftarrow \forall \Sigma_{n+1}(t_n)] \Gamma'' \\ \Sigma_{n+1}, \Gamma'' \vdash S^t : \Sigma_{n+2} \end{array}$$

$$\Sigma_0, \Gamma \vdash \{ (\mathbf{decl} \ x_1 = E_1^{t_1};)^{t'_1} \cdots (\mathbf{decl} \ x_n = E_n^{t_n};)^{t'_n} \ S^t \}^{t'} : \Sigma_{n+2}$$

For **while** statements, we simply have to add a constraint that the test condition evaluates to a **bool** type. After that we treat the while statement as a block statement.

$$\begin{array}{l} (\Sigma_0 \wedge t = t_1), \Gamma \vdash E^{t_0} : \Sigma_1 \quad (\Sigma_1 \wedge t_0 = \mathbf{bool}) = \Sigma_2 \quad \Sigma_2, \Gamma \vdash \{ S \}^{t_1} : \Sigma_3 \end{array}$$

$$\Sigma_0, \Gamma \vdash (\mathbf{while} \ (E^{t_0}) \ \{ S \}^{t_1})^t : \Sigma_3$$

For **for** statements, the typing rule is the same as typing a **while** statement, with the addition of checking the init and update nodes before checking the body.

$$\begin{array}{l} (\Sigma_0 \wedge t = t_3), \Gamma \vdash \text{assign}_1^{t_0} : \Sigma_1 \\ \Sigma_1, \Gamma \vdash E^{t_1} : \Sigma_2 \\ (\Sigma_2 \wedge t_1 = \mathbf{bool}) = \Sigma_3 \\ \Sigma_3, \Gamma \vdash \text{assign}_2^{t_2} : \Sigma_4 \\ \Sigma_4, \Gamma \vdash \{ S \}^{t_3} : \Sigma_5 \end{array}$$

$$\Sigma_0, \Gamma \vdash (\mathbf{for} \ (\ \text{assign}_1^{t_0}; \ E^{t_1}; \ \text{assign}_2^{t_2} \) \ \{ S \}^{t_3})^t : \Sigma_5$$

2.4 Typing Relation on Conditionals

For the typing of conditional expressions and conditional statements, the idea is that if the condition is a “predicate test”, then in the consequent branch, we should assert that the variable has the appropriate type. It’s easier to understand with an example:

```
function id(xs) {
  if (is_pair(xs)) {
    return pair(head(xs), tail(xs));
  } else {
    return xs;
  }
}
```

In the above example, we are allowed to give the type $\forall((T_0) \rightarrow T_0)$ to the function `id`, even though the subexpression `head(xs)` on line 3 would normally impose a type constraint on `xs` to be of the form $\text{Pair}(t_{\text{head}}, t_{\text{tail}})$.

This is because line 3 is “protected” by the predicate test `is_pair(xs)`, which allows the variable `xs` on line 3 to have the more specific type $\text{Pair}(t_{\text{head}}, t_{\text{tail}})$.

To make this tractable for type checking, “predicate tests” can only be of the form

$$x(y)$$

where both x and y are identifiers. Furthermore, x must be of a predicate type in the current type environment.

We can make this idea more concrete with the following rule for illustration purposes. The actual rules we will use are a little more complex.

$$\frac{\begin{array}{c} \Gamma(x) = \text{Pred}(t_{\text{check}}) \\ x \text{ and } y \text{ are names} \\ (\Sigma_0 \wedge t_{\text{ynew}} = \text{fresh}(t_{\text{check}})) = \Sigma_1 \quad \Sigma_1, \Gamma[y \leftarrow t_{\text{ynew}}] \vdash E_1^{t_1} : \Sigma_2 \\ \Sigma_2, \Gamma \vdash E_2^{t_2} : \Sigma_3 \\ (\Sigma_3 \wedge t_1 = t_2 \wedge t_2 = t) = \Sigma_4 \end{array}}{\Sigma_0, \Gamma \vdash ((x^{t_x}(y^{t_y}))^{t_0} ? E_1^{t_1} : E_2^{t_2})^t : \Sigma_4} \quad \begin{array}{c} \text{(for illustration} \\ \text{purposes only)} \end{array}$$

This rule is only able to handle expressions of the form $x(y) ? E_1 : E_2$, but we would like to be able to also handle conjunctions of predicate tests (possibly with other arbitrary boolean expressions) such as $x_1(y_1) \ \&\& \ E_0 \ \&\& \ x_2(y_2) \ \&\& \ x_3(y_3) ? E_1 : E_2$.

In such cases, we know that whenever the entire condition evaluates to true, then the 3 predicate tests nested within all necessarily evaluate to true. Thus, we should be able to apply all 3 predicate tests to the type environment when type checking E_1 .

It is worth noting that the special handling of certain boolean expressions of predicate tests we’re about to define have only been added for programmer convenience, as there are usually ways to “desugar” these more complex boolean expressions into nested conditional expressions in a way that results in the same predicate tests being applied.

To facilitate this, we define two functions, Extract^+ and Extract^- , which given an expression and type environment, will each return a set of predicate tests. Extract^+ will return all predicate tests that necessarily evaluate to true whenever the given expression evaluates to true. Extract^- will instead return all predicate tests that necessarily evaluate to true whenever the given expression evaluates to false.

$$\frac{\begin{array}{c} \text{where } E \text{ is neither} \\ \text{of the form } E1 \ \&\& \ E2 \\ \text{nor of the form } !E1 \\ \text{nor a predicate test in the type environment } \Gamma \end{array}}{\text{Extract}_{\Gamma}^+(E) = \emptyset}$$

$$\frac{\begin{array}{c} \text{where } E \text{ is neither} \\ \text{of the form } E1 \ || \ E2 \\ \text{nor of the form } !E1 \end{array}}{\text{Extract}_{\Gamma}^-(E) = \emptyset}$$

$$\frac{\begin{array}{c} \Gamma(x) = \text{Pred}(t_{\text{check}}) \quad x \text{ and } y \text{ are names} \end{array}}{\text{Extract}_{\Gamma}^+(x(y)) = \{(y, t_{\text{check}})\}}$$

Note that the rule above defines what is meant by predicate test.

$$\frac{\begin{array}{c} \text{Extract}_{\Gamma}^+(E_0) = S_0 \quad \text{Extract}_{\Gamma}^+(E_1) = S_1 \quad S_0 \cup S_1 = S \end{array}}{\text{Extract}_{\Gamma}^+(E_0 \ \&\& \ E_1) = S}$$

$$\begin{array}{c}
\text{Extract}_{\Gamma}^{-}(E_0) = S_0 \quad \text{Extract}_{\Gamma}^{-}(E_1) = S_1 \quad S_0 \cup S_1 = S \\
\hline
\text{Extract}_{\Gamma}^{-}(E_0 \mid \mid E_1) = S \\
\\
\text{Extract}_{\Gamma}^{-}(E) = S \quad \text{Extract}_{\Gamma}^{+}(E) = S \\
\hline
\text{Extract}_{\Gamma}^{+}(! (E)) = S \quad \text{Extract}_{\Gamma}^{-}(! (E)) = S
\end{array}$$

Now we may define the following rule for conditional expressions involving predicate tests.

$$\begin{array}{c}
\text{Extract}_{\Gamma}^{+}(E_0) = \{(y_0, t_{check_0}), (y_1, t_{check_1}), \dots\} \text{ is nonempty} \\
(\Sigma_0 \wedge t_0 = \text{b} \circ \circ \text{l}) = \Sigma_1 \quad \Sigma_1, \Gamma \vdash E_0^{t_0} : \Sigma_2 \\
(\Sigma_2 \wedge t_{newy_0} = \text{fresh}(t_{check_0}) \wedge t_{newy_1} = \text{fresh}(t_{check_1}) \wedge \dots) = \Sigma_3 \\
\Sigma_3, \Gamma[y_0 \leftarrow t_{newy_0}][y_1 \leftarrow t_{newy_1}] \dots \vdash E_1^{t_1} : \Sigma_4 \\
\Sigma_4, \Gamma \vdash E_2^{t_2} : \Sigma_5 \\
(\Sigma_5 \wedge t_1 = t_2 \wedge t_2 = t) = \Sigma_6 \\
\hline
\Sigma_0, \Gamma \vdash (E_0^{t_0} ? E_1^{t_1} : E_2^{t_2})^t : \Sigma_6
\end{array}$$

To handle situations where the condition expression has been negated, rather than giving up immediately due to having no extractable predicate tests, we will attempt to typecheck it with the consequent and alternate flipped.

$$\begin{array}{c}
\text{Extract}_{\Gamma}^{+}(E_0) \text{ is nonempty} \quad \Sigma_0, \Gamma \vdash (E_0^{t_0} ? E_2^{t_2} : E_1^{t_1})^t : \Sigma_1 \\
\hline
\Sigma_0, \Gamma \vdash (! (E_0^{t_0}) ? E_1^{t_1} : E_2^{t_2})^t : \Sigma_1
\end{array}$$

In the event that a conditional expression involves no predicate tests, we will type it the usual way with the following rule:

$$\begin{array}{c}
\text{Extract}_{\Gamma}^{+}(E_0) \text{ is empty} \quad \text{Extract}_{\Gamma}^{-}(E_0) \text{ is empty} \\
(\Sigma_0 \wedge t = t_1), \Gamma \vdash E_0^{t_0} : \Sigma_1 \quad (\Sigma_1 \wedge t_0 = \text{b} \circ \circ \text{l}) = \Sigma_2 \\
\Sigma_2, \Gamma \vdash E_1^{t_1} : \Sigma_3 \quad \Sigma_3, \Gamma \vdash E_2^{t_2} : \Sigma_4 \quad (\Sigma_4 \wedge t_1 = t_2) = \Sigma_5 \\
\hline
\Sigma_0, \Gamma \vdash (E_0^{t_0} ? E_1^{t_1} : E_2^{t_2})^t : \Sigma_5
\end{array}$$

The type of conditional statements is similar to the type of conditional expressions.

$$\begin{array}{c}
\text{Extract}_{\Gamma}^{+}(E_0) = \{(y_0, t_{check_0}), (y_1, t_{check_1}), \dots\} \text{ is nonempty} \\
(\Sigma_0 \wedge t_0 = \text{b} \circ \circ \text{l}) = \Sigma_1 \quad \Sigma_1, \Gamma \vdash E_0^{t_0} : \Sigma_2 \\
(\Sigma_2 \wedge t_{newy_0} = \text{fresh}(t_{check_0}) \wedge t_{newy_1} = \text{fresh}(t_{check_1}) \wedge \dots) = \Sigma_3 \\
\Sigma_3, \Gamma[y_0 \leftarrow t_{newy_0}][y_1 \leftarrow t_{newy_1}] \dots \vdash \{S_1\}^{t_1} : \Sigma_4 \\
\Sigma_4, \Gamma \vdash \{S_2\}^{t_2} : \Sigma_5 \\
(\Sigma_5 \wedge t_1 = t_2 \wedge t_2 = t) = \Sigma_6 \\
\hline
\Sigma_0, \Gamma \vdash (\text{if } (E_0^{t_0}) \{ S_1 \}^{t_1} \text{ else } \{ S_2 \}^{t_2})^t : \Sigma_6
\end{array}$$

$$\text{Extract}_T^+(E_0) \text{ is nonempty} \quad \Sigma_0, \Gamma \vdash (\mathbf{if} (E_0^{t_0}) \{ S_1 \}^{t_1} \mathbf{else} \{ S_2 \}^{t_2})^t : \Sigma_1$$

$$\Sigma_0, \Gamma \vdash (\mathbf{if} (E_0^{t_0}) \{ S_1 \}^{t_1} \mathbf{else} \{ S_2 \}^{t_2})^t : \Sigma_1$$

$$\begin{array}{c} \text{Extract}_T^+(E_0) \text{ is empty} \quad \text{Extract}_T^-(E_0) \text{ is empty} \\ (\Sigma_0 \wedge t = t_1), \Gamma \vdash E_0^{t_0} : \Sigma_1 \quad (\Sigma_1 \wedge t_0 = \text{b} \circ \circ 1) = \Sigma_2 \\ \Sigma_2, \Gamma \vdash \{ S_1 \}^{t_1} : \Sigma_3 \quad \Sigma_3, \Gamma \vdash \{ S_2 \}^{t_2} : \Sigma_4 \quad (\Sigma_4 \wedge t_1 = t_2) = \Sigma_5 \end{array}$$

$$\Sigma_0, \Gamma \vdash (\mathbf{if} (E_0^{t_0}) \{ S_1 \}^{t_1} \mathbf{else} \{ S_2 \}^{t_2})^t : \Sigma_5$$

3 Type Safety of Source

Now we can define what it means for a statement to be well-typed.

Definition 3.1 *A statement S is well-typed, if there is a consistent type constraint Σ such that $S : \Sigma$.*

Note that this definition of well-typedness requires that a well-typed statement has no free names.